

Decision trees

Dataset: Forest CoverType, seven species

How to use these notes

The primary text for this lecture is Géron, Chapter 5. These notes are not a replacement for it and do not repeat it: they are the lecture’s own argument written out at length — the derivation done slowly, the numbers with the runs that produced them, and the reasoning between one slide and the next that a deck can only gesture at. Read the chapter for the method; read these for why the lecture makes the choices it does. Every figure quoted here is printed by this lecture’s notebook, and the course checks that mechanically rather than trusting it.

Contents

1	A model family chosen by the shape of the answer	1
1.1	The brief	1
1.2	The data, and what we did to it	2
2	Derivation: impurity, and the objective CART minimises	3
2.1	What an impurity measure has to do	3
2.2	The objective, and why a split never hurts	4
2.3	Why the two criteria usually choose the same split	5
2.4	Measure it rather than accepting it	5
2.5	What the derivation does not give you	6
3	Growing a tree, and reading one	6
3.1	Fit one with no constraints	6
3.2	What depth actually buys	7
3.3	Overruling the grid a second time	7
3.4	Reading one prediction	8

4	One number for seven species is a summary	8
5	Regression trees, and two ways a tree breaks	9
5.1	Failure condition 1 — every boundary is axis-aligned	10
5.2	Failure condition 2 — ask a question accuracy cannot answer	10
6	The notebook	11
7	Exercises	11
	Summary	11

1 A model family chosen by the shape of the answer

Every model so far predicts with a weighted sum $\theta^T \mathbf{x}$, fitted by solving a linear system or descending a convex surface. Such models need their features scaled, because the geometry of the loss depends on the units; they reach curves only by manufacturing the curve yourself; and their explanation of a prediction is a sum of n terms — a reason, but not a short one.

Today’s model is not a weighted sum at all. It is a sequence of yes/no questions, and *the sequence is the explanation*.

1.1 The brief

An agency managing a national forest must publish a map of forest cover type — which species dominates each 30 by 30 metre patch — over an area far too large to survey on foot. From cartographic measurements alone (elevation, slope, aspect, distances to water, road and fire ignition points, hillshade, soil type) predict which of seven species covers the patch.

The map goes into the public record. It decides where logging is permitted, which parcels qualify for habitat protection, and how fire risk is modelled for the district. A landowner refused a permit is entitled to ask why. So is a court.

The requirement that decides the model

Every *individual* prediction must be accompanied by a human-readable justification: a statement, in terms of the measured quantities, of why this patch was classified as that species.

Note the word *individual*. Not “which variables matter in general”. Not “the model is 88% accurate”. This patch, this answer, this reason.

“Human-readable” is not a testable condition, so it has to be negotiated into one before any code is written.

Requirement	Testable form
Readable by a non-specialist	stated as conditions on the measured columns, not on transformed ones
Short enough to check	at most 8 conditions per prediction
Auditable	the same patch gives the same justification, and the rule can be applied by hand

Eight is the number the agency chose. It is arbitrary — and it is now a hyperparameter fixed by the brief rather than by cross-validation. That is unusual and worth noticing.

Model	Can it justify one prediction?
Logistic regression	partly — a weighted sum of 54 terms is a reason, but not a short one
k -nearest neighbours	“because these 5 other patches looked similar”
SVM with an RBF kernel	no — the decision surface has no finite description
Neural network	no
Decision tree	yes — the path from root to leaf <i>is</i> the justification

This is the first problem in the course where the model family is chosen by the shape of the answer rather than by the data.

1.2 The data, and what we did to it

581,012 patches, each with 54 measurements: 10 quantitative, 4 already one-hot for wilderness area, 40 already one-hot for soil type. No missing values, no text columns, nothing to impute. This one arrives clean, so the difficulty is entirely elsewhere.

We use 60,000 patches, stratified so the class proportions are preserved exactly, split 48,000 / 12,000.

Say what you did, and why

A 200-tree ensemble on all 581,012 rows takes minutes per fit, which does not fit inside a ninety-minute session. Every number in this lecture and the next is measured on the subsample, and every one of them would move on the full set.

Subsampling for speed is a legitimate choice and a *reportable* one. Subsampling and not mentioning it is not.

Class	Training patches	Species
2	23,405	Lodgepole Pine
1	17,501	Spruce/Fir
3	2,954	Ponderosa Pine
7	1,694	Krummholz
6	1,435	Douglas-fir
5	784	Aspen
4	227	Cottonwood/Willow

The commonest class is a hundred times the rarest, and two species are 85% of the ground. A classifier that always says “Lodgepole Pine” scores **48.8%**, so anything below that is worse than a constant and the distance above it is the only part you earned. The agency’s surveyors are stated to agree with one another on roughly nine patches in ten — an assumption we were handed, not a measurement we made — so the useful range lies between those two numbers.

2 Derivation: impurity, and the objective CART minimises

You called `DecisionTreeClassifier().fit()`. It chose one feature and one threshold at every node. Chose them to minimise *what*?

A node holds m instances over K classes, with p_k the fraction in class k . The class distribution $\mathbf{p}$ is the *only* thing an impurity measure sees. A node is **pure** when some $p_k = 1$, and **uniform** when every $p_k = 1/K$.

2.1 What an impurity measure has to do

Before choosing a formula, say what any acceptable one must satisfy:

- (i) $I(\mathbf{p}) = 0 \iff \mathbf{p}$ is pure
- (ii) I is maximal at $\mathbf{p} = (1/K, \dots, 1/K)$.

A node we have finished with scores zero; the node we know least about scores worst. Everything else is a choice *within* this specification, not a disagreement about it.

Both of Scikit-Learn’s criteria have one shape:

$$I(\mathbf{p}) = \sum_{k=1}^K \varphi(p_k),$$

with $\varphi : [0, 1] \rightarrow \mathbb{R}$ strictly concave and $\varphi(0) = \varphi(1) = 0$. Concavity is not decoration — it is the property that delivers both conditions above and the guarantee below, and it is the only property either proof uses.

Gini impurity takes $\varphi(p) = p(1 - p)$:

$$G(\mathbf{p}) = \sum_k p_k(1 - p_k) = 1 - \sum_k p_k^2.$$

Draw two of the node’s instances independently with replacement; $\sum_k p_k^2$ is the probability they are the same class, so G is the probability they *differ*. A pure node never disagrees with itself.

Entropy takes $\varphi(p) = -p \log_2 p$:

$$H(\mathbf{p}) = - \sum_k p_k \log_2 p_k,$$

the average number of bits needed to transmit the class of an instance drawn from the node, under the best possible code. A pure node needs none.

Both satisfy the specification

(i) $\varphi \geq 0$ on $[0, 1]$ and vanishes only at $p \in \{0, 1\}$, for both choices. So $I(\mathbf{p}) = 0$ forces every $p_k \in \{0, 1\}$, and since they sum to 1 exactly one is 1: the node is pure.

(ii) By Jensen's inequality on the concave φ ,

$$\frac{1}{K} \sum_k \varphi(p_k) \leq \varphi\left(\frac{1}{K} \sum_k p_k\right) = \varphi(1/K),$$

so $I(\mathbf{p}) \leq K\varphi(1/K)$. Strict concavity makes this an equality only when every p_k is equal, so the maximum is attained at the uniform distribution and nowhere else.

K	$G_{\max} = 1 - 1/K$	$H_{\max} = \log_2 K$ (bits)
2	0.5	1
7	$6/7 \approx 0.857$	$\log_2 7 \approx 2.807$

The two are on *different scales*. Any comparison of their shapes has to divide one of them first, and saying which is part of the comparison.

2.2 The objective, and why a split never hurts

A split sends m_L instances left and m_R right. Its cost is the impurity of the two children, weighted by their size:

$$J(k, t_k) = \frac{m_L}{m} I_L + \frac{m_R}{m} I_R,$$

searched over every feature k and every threshold t_k . Equivalently, maximise $\Delta I = I_{\text{parent}} - J$; the parent term is the same for every candidate at this node, so the two are the same search.

$\Delta I \geq 0$, always

Every instance goes left or right, so for each class the parent share is the size-weighted mixture of the children's shares:

$$p_k = \frac{m_L}{m} p_{L,k} + \frac{m_R}{m} p_{R,k}.$$

Apply concavity of φ to that convex combination and sum over k :

$$I(\mathbf{p}) \geq \frac{m_L}{m} I_L + \frac{m_R}{m} I_R = J.$$

So the greedy search can always stop safely: no split ever makes things worse, and $\Delta I = 0$ signals that this node has nothing left to gain.

2.3 Why the two criteria usually choose the same split

$I(\mathbf{p}) = \sum_k \varphi(p_k)$ with φ concave is **Schur-concave**: if one distribution majorises another — is more concentrated, in the sense that its largest share is larger, its two largest shares sum to more, and so on — then it has the lower impurity, *for every such φ* .

So take two candidate splits sending the same numbers left and right. If A's two children each majorise B's, then $J_A \leq J_B$ for every concave φ , and Gini and entropy must agree that A is better. Neither the choice of φ nor its scale can change it.

They are free to disagree only on pairs that are *not* comparable in that ordering. How far can they part? On a two-class node, G has maximum $1/2$ and H has maximum 1, so compare $H/2$ with G : both are 0 at $p \in \{0, 1\}$ and $1/2$ at $p = 1/2$, and the largest gap anywhere else is **0.055**, at $p \approx 0.10$ and again at $p \approx 0.90$. Two candidates whose weighted impurities differ by more than 0.055 are ranked identically by both criteria. Disagreement needs a close call *and* an incomparable pair.

2.4 Measure it rather than accepting it

Géron says entropy tends to produce slightly more balanced trees, that Gini is marginally faster, and that “most of the time it does not make a big difference”. That last is a claim about behaviour, and we have a dataset. Twenty resamples, both criteria on the same resample each time, so the comparison is paired:

Question	Measured
Resamples where the two chose the same root feature	20 of 20
Test predictions on which the two trees agree	85.3%
Mean accuracy, gini	73.4%
Mean accuracy, entropy	73.0%
Paired difference, entropy – gini	-0.44 ± 0.39 points

Exactly the shape the Schur-concavity argument predicts: the two pick the same first question every time, because the root split is not a close call, and then disagree about one prediction in seven, deep in the tree where candidates are separated by a hair.

Three true statements, and only the third is a recommendation

The effect is *real*: Gini wins on 17 of the 20 resamples and the mean sits about five standard errors from zero. The effect is *tiny*: 0.44 points, against the 8.0 points that `max_depth` is worth later in this lecture. So do not spend your tuning budget here.

“Statistically detectable” and “worth acting on” are different claims and need different evidence. Twenty paired resamples give you both; one run gives you neither.

2.5 What the derivation does not give you

$\Delta I \geq 0$ guarantees each split is locally harmless. It says nothing about the *sequence* of splits, and CART chooses that greedily: the best split at this node, never reconsidered. A split that looks mediocre now but would enable two excellent splits below it is never found.

Finding the optimal tree is NP-complete; the greedy algorithm returns a good one in $O(n m \log m)$ and no known algorithm returns the best one in reasonable time. So the tree you get is a function of the algorithm as much as of the data — the seed of this lecture's second failure condition.

3 Growing a tree, and reading one

Each internal node asks $x_k \leq t_k$ about one feature; each leaf predicts the majority class of the training instances that reached it. Predicting one instance costs $O(\log_2 m)$ — one root-to-leaf path — and does not depend on n , which is why reading a prediction costs the same as making it. That is what makes the justification free.

Trees need no feature scaling at all: $x_k \leq t_k$ is invariant under any strictly increasing transformation of x_k , so standardising a column changes its thresholds and nothing else.

3.1 Fit one with no constraints

Depth 33, 5,699 leaves, 100.0% training accuracy. With 48,000 training patches that is eight patches per leaf on average, so of course it is perfect on its own data.

Unconstrained tree	Accuracy
On the training set	100.0%
5-fold cross-validated, on the training set	82.1%
On the test set (once, at the end)	82.6%

82% is real and far above the 48.8% baseline, so the tree family works here. But count what a prediction rests on. Using `decision_path`, the mean justification is **17.88** conditions and the worst is **33**. The brief allows eight. (The maximum equalling `get_depth()` is the cheap check that the off-by-one went the right way.)

It grew to depth 33 because nothing stopped it. A tree is a **nonparametric** model: the parameter count is not fixed before training, so the structure follows the data as far as the data goes. A linear model cannot do this — its parameter count is decided by the number of columns before it sees a row.

3.2 What depth actually buys

max_depth	CV accuracy	Leaves	What a justification looks like
1	63.3%	2	one condition
3	67.5%	8	a sentence
8	73.8%	206	a short paragraph — the brief
18	81.7%	3,377	nobody reads this

Between depth 8 and depth 18 there are **8.0 points** of accuracy. That is the measured price of the eight-condition requirement, and it is a number worth putting in front of the agency.

The one hyperparameter we may not tune

Cross-validation says `max_depth=None`. The brief says `max_depth=8`. Cross-validation does not get a vote, because it is not optimising the thing the agency is buying.

This is the first time in the course that the honest answer to “what does the search say?” is “the search is answering the wrong question”. It will not be the last. Bring the 8.0-point

figure to the agency; perhaps they will trade a condition or two for it. That is a conversation, not a GridSearchCV.

We search the whole grid anyway — including the depths we may not use — because the rows we cannot pick are exactly what tells the agency what its requirement costs. Two things fall out. Within the permitted depth-8 row `min_samples_leaf` barely matters, 73.8% at 1 against 71.5% at 200, because the depth limit binds first. Down the unlimited-depth row it matters enormously, 82.1% at 1 falling to 71.9% at 200, because there the leaf size *is* the regularisation. Two hyperparameters that both restrict the tree do not act independently, and a 2-D grid shows that where two separate 1-D sweeps would not.

3.3 Overruling the grid a second time

The grid’s answer under the cap is `min_samples_leaf=1`. We ship 20 anyway.

Why

The model states its justification as “91% of the 463 training patches in this leaf”. At `min_samples_leaf=1` that becomes “100% of the 1” — a single surveyed patch wearing the grammar of evidence. The brief requires a justification that can be audited; that is not one. Cross-validation prefers 1 at 73.75%; the brief requires 20 at 73.35%. The price of auditability is 0.40 points, and it goes to the agency. A constraint that overrules a measurement has to show the measurement it overruled.

	Unconstrained	Depth 8, leaf 20
Leaves	5,699	163
Columns consulted, of 54	47	24
Mean conditions per justification	17.88	7.97
Smallest leaf	1	20
Training accuracy	100.0%	74.4%
Cross-validated accuracy	82.1%	73.4%

Notice what the last two rows now do: 74.4% against 73.4%, a gap of *one point*, where the unconstrained tree’s gap was 17.9. The depth limit did not only shorten the justification — it removed almost all of the overfitting as a side effect.

Which leaves a question worth holding on to. If the constrained tree barely overfits, why is it still nearly nine points worse than the unconstrained one?

3.4 Reading one prediction

The entire justification mechanism is `tree_.children_left`, `tree_.feature` and `tree_.threshold` — nine lines, no library beyond what is already imported. For one test patch:

```
1. Elevation = 3,763 > 3,046
2. Elevation = 3,763 > 3,306
3. Wild_2   = 1 > 0
```



```

4. Soil_31      =      0 <=      0
5. Elevation    = 3,763 > 3,360
6. HDist_Fire   = 3,020 > 364
7. Shade_Noon   = 200 <= 244
8. Shade_3pm    = 133 <= 196
-> Krummholz (91% of the 463 training patches in this leaf)

```

Eight conditions, every one a measurement a surveyor took. The path *is* the justification: no extra machinery, no approximation, no second model explaining the first.

A note on thresholds: CART puts the split midway between two adjacent observed values, which is why the root reads 3046.5 rather than 3046. Nothing in the data sits at 3,046.5 metres.

Read the leaf carefully

91% of the 463 training patches in that leaf were Krummholz. That is a statement about 463 *training patches*, not a probability that this patch is Krummholz.

`predict_proba` returns exactly those leaf proportions, so a tree’s “probabilities” are piecewise constant and identical for every patch reaching the same leaf. Across the 163 leaves the smallest holds 20 patches and the largest several thousand; those deserve very different amounts of trust, and the count belongs in the justification.

The honest sentence is “of the training patches that satisfied these eight conditions, 91% were Krummholz” — not “this patch is 91% likely to be Krummholz”.

4 One number for seven species is a summary

The test set, once: **73.0%** on 12,000 patches, against a constant baseline of 48.8%, with every choice made on cross-validated folds of the training set.

Cover type	Share of the data	Recall
Lodgepole Pine	48.8%	82.0%
Ponderosa Pine	6.2%	80.8%
Spruce/Fir	36.5%	69.6%
Krummholz	3.5%	56.4%
Cottonwood/Willow	0.5%	56.1%
Douglas-fir	3.0%	12.8%
Aspen	1.6%	2.6%

The model finds fewer than 3 Aspen patches in every hundred. On the headline number that costs 1.6 points and is invisible.

Why the tree cannot see Aspen

J weights each child’s impurity by *how many instances it holds*. A split that isolates a class at 1.6% of the node moves the weighted average by almost nothing, so the greedy search never chooses one. With `max_depth=8` there are at most 256 leaves to allocate across seven species, and the algorithm spends them where the instances are.

This is Lecture 3’s lesson in a new costume: an objective that averages over instances rewards

ignoring the rare class. It is a property of the objective, not a bug in the implementation — read the definition of J again and you could have predicted it.

`class_weight="balanced"` multiplies each instance's contribution by the inverse of its class frequency. Aspen recall rises to most of the class, because the split that isolates it is now worth choosing; overall accuracy falls sharply, because the two species that are 85% of the ground now carry 2/7 of the weight instead of 85% of it. A different model answering a different question — and quoting either number without the other is an argument rather than a measurement.

The sentence the agency needs

“This map is 73.0% accurate overall, but it identifies fewer than three Aspen stands in a hundred. Do not use it to decide anything about Aspen or Douglas-fir.”

A model can be fit for one purpose and unfit for another in the same deployment. Saying which is which is the job.

5 Regression trees, and two ways a tree breaks

`DecisionTreeRegressor` is the same algorithm with the impurity replaced by the mean squared error of the node about its own mean. A leaf predicts the mean of the training targets that reached it, so the output is a step function, one step per leaf. An unconstrained regression tree drives its training error to exactly zero. And a tree *cannot extrapolate*: beyond the range of the training targets it returns the nearest leaf's mean forever, which is the thing to know before using one on a trending series.

5.1 Failure condition 1 — every boundary is axis-aligned

Every split is $x_k \leq t_k$, so every decision boundary is perpendicular to an axis. A boundary that genuinely runs at 45° has to be approximated by a staircase, and each step costs a level of depth. Rotate the data by 45° and the same problem needs a much deeper tree, though nothing about the problem changed.

Scaling a column is harmless — it is monotone in that column alone. Rotating the space is not: it mixes columns, and the family of candidate splits is not rotation-invariant.

5.2 Failure condition 2 — ask a question accuracy cannot answer

You have a model whose every prediction comes with a rule a person can read. Fit it again — same hyperparameters, same seed — on 90% of the same training set. How similar are the two sets of rules? Almost everybody expects “very”. The answer has two halves and they point in opposite directions.

Over 20 refits	Measured
Accuracy, mean	72.99%
Accuracy, standard deviation	0.25%
Accuracy, range	72.39% – 73.38%
Pairwise prediction disagreement, over all 190 pairs	9.1%
worst pair	15.4%
Test patches all twenty trees agree on	71.4%

Two trees with the same accuracy disagree about the species of one patch in eleven. And where in the tree does it change?

Top of the tree	Across the 20 refits
Feature at the root	Elevation, 20 times out of 20
Distinct thresholds at the root	4, spanning 15 metres of elevation
Distinct structures in the top three levels	11 of 20
Leaves in the whole tree	151 to 170
Columns the tree consults	22 to 27 of 54

The part you put on a slide is stable. The part that decides is not.

A tree is a hierarchy. The root split is chosen from 48,000 patches and wins by a wide margin, so it is stable — which is also why Gini and entropy agreed about it 20 times out of 20. A node eight levels down was chosen from a few hundred patches, where two candidates are often separated by a hair; change one row, the loser becomes the winner, and everything below is a different tree.

Accuracy is an average over 12,000 patches, and averages hide substitutions: a tree that loses a patch here and gains one there scores the same. *A stable metric is not evidence of a stable model. It is evidence that you measured the wrong thing.*

This is *variance* in the sense of Lecture 5's decomposition — the sensitivity of the fitted model to the particular training sample. Trees have it badly for a structural reason: the greedy, hierarchical fit turns an arbitrarily small change in one split into an arbitrarily large change in the subtree below it, and nothing smooths it out.

Failure condition 1 was the same fact in a different hat. Rotating the data changes which axis-aligned split wins by a hair, and the tree below it changes completely.

The next lecture derives what averaging does to variance and builds the models that exploit it. It also shows what the repair costs: the justification we spent this lecture earning.

6 The notebook

What to change

Refit the shipped tree on 90% subsamples with a different set of seeds and recompute the pairwise disagreement. If it lands near 9% again, you have reproduced the instability rather than read about it — and you can then ask the harder question: which of the 12,000 patches

are the unstable ones, and do they have anything in common?

7 Exercises

1. State the two conditions any impurity measure must satisfy, and prove that entropy attains its maximum only at the uniform distribution. [5]
2. Show that a split can never increase the size-weighted impurity, naming the property of φ the argument uses. [4]
3. Two candidate splits send the same numbers left and right, and each of A's children majorises the corresponding child of B. State which criterion prefers A, and why the answer does not depend on the choice between Gini and entropy. [4]
4. A tree achieves 73.0% accuracy but 2.6% recall on a class holding 1.6% of the data. Explain, from the definition of the split objective, why this is expected rather than a defect. [5]
5. Twenty trees fitted on 90% subsamples of one training set have accuracies within 0.5 points of each other but disagree on 9% of test predictions. Explain how both can be true, and say what it tells you about reporting accuracy alone. [4]

Summary

The model family was chosen by the shape of the required answer rather than by the data: only a tree can hand a court an individual justification in the measured columns. Impurity is $\sum_k \varphi(p_k)$ for strictly concave φ , which is the only property the theory uses — it gives zero exactly at purity, a maximum exactly at uniformity by Jensen, and $\Delta I \geq 0$ for every split by the same concavity. Schur-concavity explains why Gini and entropy agree wherever the comparison is not a close call, and the measurement agreed: the same root feature 20 times out of 20, and 0.44 points between them against the 8.0 that depth is worth.

The brief fixed `max_depth=8`, which cost 8.0 points, and auditability fixed `min_samples_leaf=20`, which cost 0.40 more. Both figures go to the agency, because a constraint that overrules a measurement has to show the measurement it overruled. The shipped tree scores 73.0% against a 48.8% constant, with a mean justification of 7.97 conditions.

Then two failure conditions, both properties of the algorithm. Every boundary is axis-aligned, so a 45° boundary costs depth. And twenty refits on 90% subsamples held accuracy to within a quarter of a point while disagreeing on 9.1% of predictions — variance, in Lecture 5's sense, arriving as a stable metric hiding an unstable model.